

Portraits de Reconnaissance Faciale Automatisée : Méthodes Deep Learning pour l'Analyse des Traits

INES KHELIFI

Compiled July 17, 2022

Deux des aspects importants dans le cadre de recherche sur la reconnaissance faciale par ordinateur sont abordés ici : la détection des visages et des caractéristiques faciales d'une part, et les méthodes de comparaison et d'identification des visages d'autre part. Les meilleurs résultats apportés au problème de reconnaissance faciale sont obtenus avec les méthodes deep learning notamment avec les classifieurs SVM. Dans cette approche, la détection globale des visages, la localisation des caractéristiques faciales et la comparaison des visages sont effectuées en une seule étape. Ce rapport décrit la mise en place d'une application de détection faciale et d'analyse de données. D'une part, nous décrivons une technique d'apprentissage visuel et la manière avec laquelle on peut l'appliquer à la détection de visage dans diverses conditions. D'autre part, un algorithme de tri et d'analyse des données est implémenté afin de présenter des résultats en temps réel et permettre leur analyse statistique. Enfin, nous apportons une extension au projet en connectant la base de données à la plateforme Cloud via les connecteurs GDBC de Confluent¹ qui permettent d'automatiser l'analyse des données en temps réel (voir article en annexe²). © 2022 iTeam university

1. INTRODUCTION

La détection faciale consiste à la localisation d'un visage dans une image sans identification. Bien que de nombreux algorithmes de détection de visage existaient avant 2001, une percée majeure dans la détection faciale est apparue avec l'article de Viola-Jones "Rapid Object Detection using a Boosted Cascade of Simple Features" [1, 2].

Contrairement aux méthodes de détection faciale précédentes qui reposaient sur l'analyse des pixels, Viola-Jones a conçu un algorithme appelé "Haar-classifier" qui s'appuyait sur des fonctionnalités de type Haar. Le classificateur de Haar est un algorithme d'apprentissage automatique formé avec de nombreux échantillons positifs et négatifs pour détecter des objets dans des images. Pour que le classificateur fonctionne correctement, la taille de l'image dans l'ensemble d'apprentissage doit être la même que la taille de l'image d'entrée utilisée pour la détection des objets.

Pour une reconnaissance réussie, l'ensemble doit être robuste, c'est-à-dire qu'il doit contenir une variété d'images telles que des

images faciales (échantillons positifs) ainsi que des images non faciales (échantillons négatifs) telles que des voitures, des arbres, etc. De plus, l'ensemble doit contenir une variation d'images faciales, où le sujet regarde vers le haut ou vers le bas, avec différentes variations des expressions du visage et des conditions d'éclairage. Il est important d'avoir un ensemble varié et diversifié plutôt qu'un grand nombre d'images avec peu ou pas de variation.

2. RAPPEL SUR LES ALGORITHMES

A. Algorithmes basés sur l'apparence

Les algorithmes basés sur l'apparence utilisent les données de pixels d'image dans leur ensemble pour la reconnaissance.

La corrélation directe, Eigen-face et Fisher-Face appartiennent à cette classe d'algorithmes. La corrélation directe utilise la comparaison directe des pixels d'image de deux images faciales, produisant un score de similarité [3, 4]. Contrairement à une méthode de corrélation directe qui utilise des images faciales dans leur espace d'image d'origine, les algorithmes Eigen-face et Fisher-face réduisent l'image au facteur le plus discriminant et effectuent la comparaison l'images dans un espace de dimension réduite [5, 6].

B. Algorithmes basés sur l'apparence active

Les algorithmes basés sur l'apparence active contiennent des informations statistiques sur une forme d'image et une variation de texture. Foulques et al. [7] ont appliqué une analyse en composantes principales (PCA) pour générer un modèle statistique qui localise les points de référence sur un ensemble d'images d'apprentissage. Les points de référence sont utilisés pour entraîner le modèle à apprendre la translation entre le paramètre du modèle synthétisé et les images test. Pour trouver les paires d'image et les faire correspondre entre eux, l'erreur résiduelle du modèle est mesurée par rapport aux changements prédits des paramètres actuels du modèle ce qui conduit à un meilleur ajustement et à une meilleure reconnaissance [8].

C. Support Vector Machines

Les machines à vecteurs de support utilisent un ensemble d'images d'apprentissage pour calculer l'hyperplan de séparation optimal. Guo et al. [9] ont appliqué cette méthode à la reconnaissance faciale à l'aide d'une classification arborescente binaire, où les images faciales sont classées de manière itérative comme appartenant à l'une des deux classes qui se propagent dans une structure arborescente binaire jusqu'à ce qu'une déci-

sion finale de classification puisse être prise.

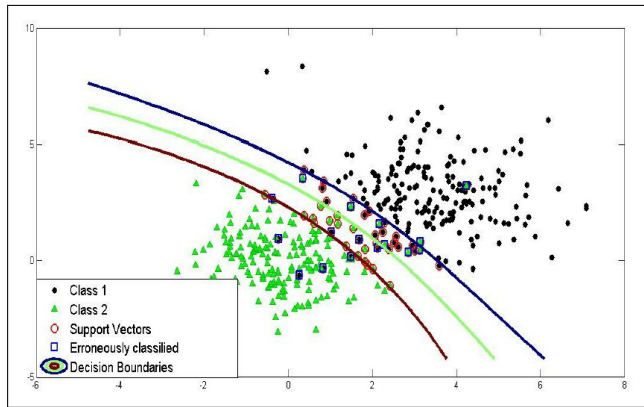


Fig. 1. SVM Classifier.

D. Algorithmes du modèle Bayésien

Les algorithmes du modèle bayésien montrent une mesure probabiliste de similarité dérivée d'une analyse bayésienne de la différence entre les images faciales. Le calcul des fonctions de probabilité utilisant les différences d'intensité d'image entre ensembles de données conduit à un score de similarité à partir de la règle de Bayes, qui peut ensuite être utilisé pour la classification de reconnaissance faciale [10].

E. Neural Networks

Les réseaux de neurones assurent le traitement de l'information, qui est similaire à la manière dont l'information est traitée dans des systèmes biologiques tels que le cerveau humain. Leur principale force est la capacité d'apprendre à partir d'exemples, la tolérance aux pannes et la robustesse. Ils sont adaptés à la reconnaissance de visages qui présentent beaucoup de variation et nécessitent cependant peu de modifications de l'algorithme de reconnaissance. Lawrence [11] décrit comment entraîner un classificateur de réseau neuronal pour l'identification et la reconnaissance faciale.

F. Algorithmes Basés sur la texture

Les algorithmes basés sur la texture extraient la texture dans les images, en divisant le visage en plusieurs régions. Le modèle binaire local (LBP) est un exemple d'algorithmes basés sur la texture, où les caractéristiques LBP pondérées sont extraites pour générer un vecteur de différentes caractéristiques. Deux vecteurs de caractéristiques LBP sont appariés en appliquant une mesure de distance Chi-carré pondérée [12].

G. Algorithmes Basés sur les caractéristiques

Ces algorithmes extraient un ensemble de caractéristiques géométriques tel que la distance à partir des images faciales et utilisent ces caractéristiques comme une référence pour comparer les images. L'analyse des caractéristiques locales est un exemple d'algorithmes basés sur les caractéristiques [13].

3. RECONNAISSANCE FACIALE AVEC KERAS

Dans ce chapitre on utilise un réseau de neurones convolutif (CNN) pour extraire les caractéristiques des images données en entrée. Le réseau suit l'approche décrite dans [14] avec des modifications inspirées du projet OpenFace. Keras est utilisé pour implémenter CNN, Dlib et OpenCV pour aligner les visages sur les images d'entrée. Les performances de reconnaissance faciale sont évaluées sur un petit sous-ensemble de l'ensemble de données LFW, qu'il est possible de remplacer par un ensemble de données personnalisé, par exemple, des images d'élèves inscrits dans l'université (pour une vérification biométrique à l'entrée).

A. Exemple d'implémentation de l'algorithme en utilisant SVM

Dans l'exemple de la figure 3, on utilise le classificateur SVM pour tester la reconnaissance faciale

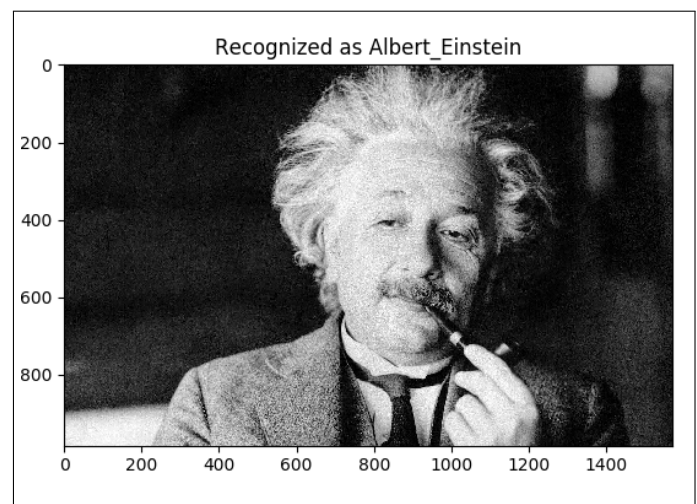


Fig. 2. Image de Albert Einstein identifiée à l'aide de l'algorithme SVM.

4. ARCHITECTURE CNN ET ENTRAÎNEMENT

L'architecture CNN utilisée ici est une variante de l'architecture d'inception [15]. Plus précisément, il s'agit d'une variante de l'architecture CNN décrite dans [14] et identifiée comme modèle nn4.small2 qu'on nomme ici model.cnn dans le projet OpenFace. Dans ce chapitre, on se charge d'une implémentation Keras de ce modèle dont la définition est tirée du projet Keras-OpenFace. Les détails de l'architecture ne sont pas détaillés ici, il est seulement utile de savoir qu'il existe une couche entièrement connectée avec 128 unités cachées suivie d'une couche de normalisation L2 au-dessus de la base convolutive. Ces deux couches supérieures sont appelées la couche d'enrobage à partir de laquelle les vecteurs d'enrobage à 128 dimensions peuvent être obtenus. Le modèle complet est décrit dans model.py.

La formation de modèle vise à apprendre une incorporation $f(x)$ d'une image x telle que la distance L2 au carré entre tous les visages de la même identité est petite et la distance entre une paire de visages d'identités différentes est grande. Ceci peut être réalisé avec une *perte de triplet* L qui est minimisée lorsque la distance entre une image d'ancrage x_i^a et une image positive x_i^p

(de la même identité) dans l'espace d'intégration est inférieure à la distance entre cette image d'ancrage et une image négative x_i^n (identité différente) d'au moins une marge α .

$$L = \sum_{i=1}^m [\|f(x)_i^a - f(x)_i^p\|_2^2 - \|f(x)_i^a - f(x)_i^n\|_2^2 + \alpha]_+ \quad (1)$$

Durant la phase d'entraînement, il est important de sélectionner les triplets dont les paires positives (x_i^a, x_i^p) et négatives (x_i^a, x_i^n) sont difficiles à discriminer. C'est-à-dire que leur différence de distance dans l'espace d'encastrement doit être inférieure à la marge α sinon, le réseau est incapable d'apprendre une intégration utile. Par conséquent, chaque itération de formation doit sélectionner un nouveau lot de triplets en fonction des intégrations apprises lors de l'itération précédente.

A. Seuil de la distance

Pour trouver la valeur optimale de τ , la vérification des visages doit être évaluée sur une intervalle de distances. À une valeur de seuil donné, toutes les paires de vecteurs d'intégration possibles sont classées soit ayant la même identité ou soit ayant une identité différente et le tout comparé au résultat attendu. Puisque nous avons affaire à des classes asymétriques (beaucoup plus de paires négatives que de paires positives), nous utilisons le score F1 comme métrique d'évaluation au lieu de la précision.

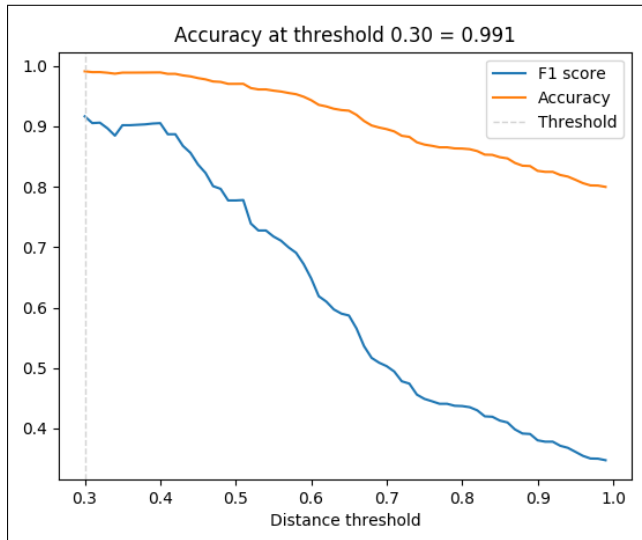


Fig. 3. Distance Threshold.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} = \frac{TP}{TP + \frac{1}{2} \times (FP + FN)} \quad (2)$$

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \quad (3)$$

L'algorithme de reconnaissance faciale ici présente une précision de 0.991 à $\tau = 0.30$. Ce n'est pas mal étant donné une ligne de base de 0.89 pour un classificateur qui prédit toujours une identité différente (il y

a 450 paires positives et 3000 paires négatives), mais puisque nn4.small2.v1 est un modèle relativement petit, il est toujours inférieur à ce qui peut être atteint par les meilleurs modèles (> 0.99).

B. Classification KNN

Étant donné une estimation du seuil de distance τ , la reconnaissance faciale est maintenant aussi simple que de calculer les distances entre un vecteur d'intégration donné à l'entrée et tous les vecteurs d'intégration dans une base de données. L'image donnée à l'entrée se voit attribuer une étiquette (c'est-à-dire une identité) d'entrée de la base de données avec la distance la plus petite si elle est inférieure à τ ou une étiquette inconnue dans le cas contraire.

Une approche plus robuste consiste à étiqueter les données à l'entrée en utilisant les entrées les mieux notées dans la base de données, ce qui représente essentiellement une classification KNN avec une métrique de distance euclidienne. Alternative-ment, une machine à vecteurs de support linéaire (SVM) peut être entraînée avec les entrées de la base de données et utilisée pour classer, c'est-à-dire identifier de nouvelles entrées. Pour la formation de ces classificateurs, nous utilisons 50 pourcents de l'ensemble de données et 50 pourcents pour l'évaluation.

C. KNN Algorithm

Algorithm 1. KNN algorithm

```

1: procedure KNN( $A, B, a, b$ ) ▷ train and test data
2:    $knn \leftarrow KNeighborsClassifier(nneighbors = 1, metric = 'euclidean')$ 
3:    $svc \leftarrow LinearSVC()$ 
4:    $mod1 \leftarrow knn.fit(A, a)$ 
5:    $mod2 \leftarrow svc.fit(A, a)$ 
6:    $KNN \leftarrow accuracy.score(b, knn.predict(B))$ 
7:    $SVC \leftarrow accuracy.score(b, svc.predict(B))$ 
8:   return KNN, SVC ▷ Accuracy Scores

```

$A : X.train, B : X.test, a : y.train, b : y.test$

D. Résultats

Dans notre exemple on voit que le classificateur KNN atteint une précision de 0.96 sur l'ensemble de test alors que le classificateur SVM atteint une précision de 0.98.

Bien que les deux algorithmes produisent des résultats satisfaisants en terme de score et précision. L'algorithme SVM offre un meilleur résultat avec une vitesse de classification nettement supérieure au KNN.

En effet, l'algorithme KNN utilise un système de vote pour déterminer à quelle classe appartient un objet non classé, en tenant compte de la classe de ses plus proches voisins dans l'espace de décision. L'algorithme SVM par contre est extrêmement rapide, capable de classer des images aériennes de 12 mégapixels en environ dix secondes (contrairement au KNN qui prend entre quarante et cinquante secondes). Généralement, l'algorithme KNN classe avec précision; cependant, il génère plusieurs petites erreurs de classification qui interfèrent avec l'image classifiée finale. Par contre, l'algorithme

SVM classera parfois de manière erronée un objet volumineux qui interfère rarement avec l'image classifiée au final.

5. LIVE CAMERA RECOGNITION

Dans la première partie on implémente une reconnaissance faciale en direct avec un système de surveillance en temps réel (caméra placé à l'entrée du bâtiment). Pour cela on utilise les bibliothèques face-recognition et OpenCv2.

Les données utilisées à l'entrée sont décrites dans database-students.csv et comprennent le nom, prénom, âge, spécialité etc ... de chaque étudiant.

Ces données sont analysées avec l'algorithme OpenCv2 d'une part et face-recognition d'autre part pour obtenir une reconnaissance faciale sur une caméra en marche.

Il est à noter que parmi les inconvénients de OpenCv2 est qu'il utilise un moyen de calcul qui devient de moins en moins rapide si on utilise une large base de donnée dans la phase d'entraînement.

Néanmoins cela reste un algorithme assez simple et rapide à implémenter avec diverses modules et méthodes de calcul pour effectuer la reconnaissance faciale.

A. Description algorithmique de la méthode EigenFaces

Soit l'ensemble $X = \{x_1, x_2, \dots, x_n\}$ un vecteur aléatoire avec des observations $x_i \in \mathbb{R}^d$.

1. Calcul de la moyenne μ :

$$\mu = \frac{1}{n} \sum_{i=1}^m x_i \quad (4)$$

2. Calcul de la matrice de covariance S :

$$S = \frac{1}{n} \sum_{i=1}^m (x_i - \mu)(x_i - \mu)^T \quad (5)$$

3. Calcul des eigenvalues λ_i et eigenvectors v_i de S :

$$Sv_i = \lambda_i v_i, i = 1, 2, \dots, n \quad (6)$$

4. Classification des vecteurs propres (eigenvectors) de manière descendante selon leurs valeurs propre (eigenvalues). Les κ composantes principales sont les vecteurs propres correspondant aux κ plus grandes valeurs propres. Les κ composantes principales du vecteur observé x sont alors données par :

$$y = W^T(x - \mu) \quad (7)$$

où $W = (v_1, v_2, \dots, v_k)$.

La reconstruction par PCA est donnée par :

$$x = Wy + \mu \quad (8)$$

où $W = (v_1, v_2, \dots, v_k)$.

La méthode Eigenfaces effectue ensuite la reconnaissance faciale par :

- Projection de tous les échantillons d'apprentissage sur l'espace PCA.

- Projection de l'image test sur l'espace PCA.

- Recherche du voisin le plus proche entre les images d'entraînement projetées et l'image test.

Il reste cependant un problème à résoudre. Dans le cas où l'on reçoit 400 images de 100×100 pixels. L'analyse en composantes principales résout la matrice de covariance $S = XX^T$, où la taille est $size(X) = 10000 \times 400$ dans notre exemple.

On se retrouve avec une matrice 10000×10000 , soit environ 0,8 Go. Résoudre ce problème n'est pas faisable, mais on peut s'y prendre autrement : On sait qu'une matrice $M \times N$ avec $M > N$ ne peut avoir que $N - 1$ valeurs propres non nulles. Il est donc possible d'utiliser à la place la décomposition en valeurs propres $S = X^T X$ de taille $N \times N$:

$$X^T X v_i = \lambda_i v_i \quad (9)$$

Et obtenir les vecteurs propres originaux de $S = XX^T$ avec une multiplication à gauche de la matrice de données :

$$XX^T (Xv_i) = \lambda_i (Xv_i) \quad (10)$$

Les vecteurs propres résultants sont orthogonaux, pour obtenir des vecteurs propres orthonormés, ils doivent être normalisés à la longueur unitaire.

B. Description algorithmique de la méthode Fisherfaces

Soit X un vecteur aléatoire avec un échantillon de c classes :

$$X = \{X_1, X_2, \dots, X_c\} \quad (11)$$

$$X_i = \{x_1, x_2, \dots, x_n\} \quad (12)$$

Les Matrices de diffusion S_B et S_W sont calculé comme suit :

$$S_B = \sum_{i=1}^c N_i (\mu_i - \mu)(\mu_i - \mu)^T \quad (13)$$

$$S_W = \sum_{i=1}^c \sum_{x_j \in X_i} (x_j - \mu_i)(x_j - \mu_i)^T \quad (14)$$

où μ représente la moyenne totale :

$$\mu = \frac{1}{N} \sum_{i=1}^N x_i \quad (15)$$

Et μ_i représente la moyenne de la classe $i \in \{1, \dots, c\}$:

$$\mu_i = \frac{1}{|X_i|} \sum_{x_j \in X_i} x_j \quad (16)$$

L'algorithme classique de Fisher recherche maintenant une projection W , qui maximise le critère de séparation des classes :

$$W_{opt} = \underset{W}{\operatorname{argmax}} \frac{|W^T S_B W|}{|W^T S_W W|} \quad (17)$$

Une solution à ce problème d'optimisation consiste à résoudre le Problème Général des eigen values:

$$S_B v_i = \lambda_i S_W v_i \quad (18)$$

$$S_W^{-1} S_B v_i = \lambda_i v_i \quad (19)$$

Il reste un problème à résoudre : le rang de S_W est au plus $(N - c)$, avec N échantillons et c classes. Dans les problèmes de reconnaissance de formes, le nombre d'échantillons N est presque toujours inférieur à la dimension des données d'entrée (nombre de pixels), de sorte que la matrice de dispersion S_W devient singulière comme dans l'exemple de [16]. Dans [6], cela a été résolu en effectuant une analyse en composantes principales sur les données et en projetant les échantillons dans l'espace de dimension $(N - c)$. Une analyse par discrimination linéaire a ensuite été effectuée sur les données réduites, car S_W ne présente plus de singularité.

Le problème d'optimisation peut être défini comme suit :

$$W_{pca} = \operatorname{argmax}_w |W^T S^T W| \quad (20)$$

$$W_{fld} = \operatorname{argmax}_w \frac{|W^T W_{pca}^T S_B W_{pca} W|}{|W^T W_{pca}^T S_W W_{pca} W|} \quad (21)$$

La matrice de transformation W qui projette un échantillon dans un espace de dimension $(c - 1)$ est donné par :

$$W = W_{fld}^T W_{pca}^T \quad (22)$$

C. Description algorithmique de la méthode LBP

Une description plus formelle de l'opérateur peut être représentée par :

$$LBP_{(x_c, y_c)} = \sum_{p=0}^{p-1} 2^p s(i_p - i_c) \quad (23)$$

avec (x_c, y_c) comme pixel central avec une intensité i_c , et i_n l'intensité du pixel avoisinant. s représente la fonction signal définie par :

$$s(x) = \begin{cases} 1 & \text{if } x \geq 0 \\ 0 & \text{else} \end{cases}$$

Cette description nous permet de capturer des détails très fins dans les images. L'article [12] a introduit une nouvelle idée qui consiste à aligner un nombre arbitraire de voisins sur un cercle de rayon variable qui permet de capturer le voisinage de manière suivante :

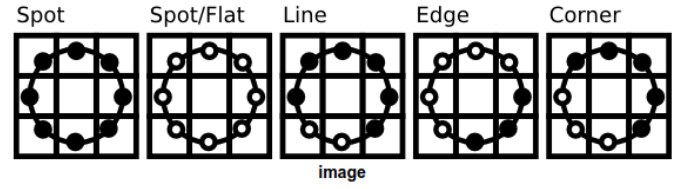


Fig. 4. Alignement de pixels sur un cercle

Pour un point donné (x_c, y_c) la position du voisin (x_p, y_p) , $p \in P$ peut être calculée par :

$$x_p = x_c + R \cos\left(\frac{2\pi p}{P}\right) \quad (24)$$

$$y_p = y_c + R \sin\left(\frac{2\pi p}{P}\right) \quad (25)$$

où R représente le rayon du cercle et P le nombre de points pris comme échantillon.

L'opérateur est une extension des codes LBP, il est donc parfois appelé "Extended LBP" (ou LBP circulaire). Si certaines coordonnées des points sur le cercle ne correspondent pas aux coordonnées de l'image, le point obtenu est interpolé. L'implémentation OpenCV par exemple fait une interpolation bilinéaire :

$$f(x) \approx [x - 1, x] \begin{bmatrix} f(0,0) & f(0,1) \\ f(1,0) & f(1,1) \end{bmatrix} \begin{bmatrix} 1 - y \\ y \end{bmatrix}$$

Par définition, l'opérateur LBP est robuste contre les transformations monolithiques sur les échelles de gris. On peut facilement vérifier cela en examinant une image LBP extraite d'une image modifiée artificiellement.

Il reste donc à déterminer comment intégrer l'information spatiale dans le modèle de reconnaissance faciale. La représentation proposée par Ahonen et. al [12] consiste à diviser l'image LBP en m régions locales et à en extraire un histogramme. Le vecteur caractéristique amélioré spatialement est alors obtenu en concaténant les histogrammes locaux (sans les fusionner). Ces histogrammes sont appelés histogrammes locaux de motifs binaires.

6. EN RÉSUMÉ

Le choix des bibliothèques de deep learning est important car les performances de l'algorithme varient beaucoup avec la quantité de donnée. On a observé dans notre cas que pour un échantillon de 200 personnes enregistrés dans la base, on a une meilleure identification avec SVM qu'avec KNN avec une vitesse de calcul de l'algorithme presque identique.

La reconnaissance faciale en direct (live caméra) a l'avantage de dresser le profil d'une personne instantanément et peut être utilisée dans le cas où l'on a une visibilité directe et à courte distance

7. CONCLUSION D'ÉTUDE

Le but des algorithmes de reconnaissance faciale consiste à dériver un ensemble de caractéristiques à partir des images qui minimise des variations intra-classe (se trouvant dans les instances d'un même individu) et maximise des variations extra-classe (images d'individus différents) [17, 18]. Si les méthodes de calculs pour classer les images sont mal choisies, la performance du classifieur est considérablement impactée. Par conséquent, il est important de décider avec soin de la méthode à adopter lors de la conception de systèmes de reconnaissance d'image.

Idéalement, la représentation des traits devrait : (i) bien discriminer les images tout en tolérant les variations intra-classe ; (ii) être facilement extraites afin de permettre un traitement rapide ; et (iii) se situer dans un espace de faible dimension (courte distance vectorielle) afin d'éviter des calculs coûteux.

Naturellement, il n'est pas facile de travailler dans des conditions idéales où le classificateur peut performer avec une précision maximale de 100%. En effet, il est toujours important de tenir compte de la variabilité des différents facteurs d'imagerie lors de la capture des images tels que l'échelle, l'orientation, la pose, les expressions faciales, les conditions d'éclairage, la distance, etc... Il s'agit aussi d'autres critères aussi importants tels que : la détection, la localisation, l'enregistrement, et la vérification.

De nombreuses méthodes ont été proposées pour palier à ces problèmes. Les premières tentatives, comme les travaux de Kanade au début des années 70 [19], sont basées sur la représentation des visages en termes de caractéristiques géométriques, telles que les distances et les angles entre les repères etc ... Plus tard, des techniques basées sur l'apparence ont été proposées : Ces méthodes représentent généralement le visage comme un tableau 2D de pixels et visent à dériver des caractéristiques d'apparence sans avoir nécessairement recours à des calculs géométriques.

De même, différentes méthodes holistiques telles que l'analyse en composantes principales (ACP) [20], l'analyse discriminante linéaire (LDA) [21] et l'ACP 2D plus récemment [22] ont été mises en place.

Les méthodes se basant sur les descripteurs locaux ont suscité une attention croissante en raison de leurs robustesse vis à vis des changements de pose et d'éclairage. Parmi ces descripteurs figurent les filtres de Gabor et les modèles binaires locaux [23] qui se sont avérés très efficaces sur le terrain.

8. OUVERTURE ET PERSPECTIVES

Avec l'approfondissement de la recherche sur les réseaux neuronaux et l'apprentissage machine, la reconnaissance faciale est devenue une technique très utilisée dans différents aspects du quotidien. Les algorithmes sont assez performant à petite échelle et on peut atteindre des niveaux de précisions de 99% dans la plupart des cas. Par contre avec l'avènement de l'ère des méga données, les méthodes de reconnaissance faciale à petite échelle ne sont plus en mesure de performer de manière efficace et c'est pour cette raison que l'intérêt de développer des algorithmes de reconnaissance d'image permettant l'intégration de données volumineuses en temps réel est devenu le nouveau point central de la recherche en intelligence artificielle.

REFERENCES

1. P. A. Viola and M. J. Jones, .
2. R. Lienhart and J. Maydt, .
3. R. Brunelli and T. Poggio, **15**, 1042.
4. A. V. Nefian, "Statistical approaches to face recognition," .
5. M. Turk, "A random walk through eigenspace," .
6. P. N. Belhumeur, J. P. Hespanha, and D. J. Kriegman, **19**, 10.
7. T. Cootes, K. Walker, and C. Taylor, "View-based active appearance models," in *Proceedings Fourth IEEE International Conference on Automatic Face and Gesture Recognition (Cat. No. PR00580)*, pp. 227–232.
8. W. Zhao, R. Chellappa, P. J. Phillips, and A. Rosenfeld, **35**, 399.
9. G. Guo, S. Li, and K. Chan, .
10. B. Moghaddam, T. Jebara, and A. Pentland, **33**, 1771.
11. S. Lawrence, C. Giles, A. C. Tsoi, and A. Back, **8**, 98. Conference Name: IEEE Transactions on Neural Networks.
12. T. Ahonen, A. Hadid, and M. Pietikainen, **28**, 2037. Conference Name: IEEE Transactions on Pattern Analysis and Machine Intelligence.
13. P. S. Penev and J. J. Atick, **7**, 477. Publisher: Taylor & Francis _eprint: https://doi.org/10.1088/0954-898X_7_3_002.
14. F. Schroff, D. Kalenichenko, and J. Philbin, "FaceNet: A unified embedding for face recognition and clustering," in *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 815–823.
15. C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich, "Going deeper with convolutions," Type: article.
16. S. Raudys, **13**, 252.
17. A.-R. Sadeghi, T. Schneider, and I. Wehrenberg, "Efficient privacy-preserving face recognition," in *Information, Security and Cryptology – ICISC 2009*, D. Lee and S. Hong, eds. (Springer), Lecture Notes in Computer Science, pp. 229–244.
18. P. Viola and M. Jones, "Robust real-time object detection," .
19. S. N. Patel, J. W. Summet, and K. N. Truong, "BlindSpot: Creating capture-resistant spaces," in *Protecting Privacy in Video Surveillance*, A. Senior, ed. (Springer), pp. 185–201.
20. M. Turk and A. Pentland, "Face recognition using eigenfaces," in *1991 IEEE Computer Society Conference on Computer Vision and Pattern Recognition Proceedings*, pp. 586–591. ISSN: 1063-6919.
21. M. Lades, J. Vorbruggen, J. Buhmann, J. Lange, C. von der Malsburg, R. Wurtz, and W. Konen, **42**, 300. Conference Name: IEEE Transactions on Computers.
22. G. Schwarzer, **71**, 391. Publisher: [Wiley, Society for Research in Child Development].
23. V. Starovoitov and D. Bryliuk, "Access control by face recognition using neural networks," .